

1 Name your Jupyter Notebook as

TASK1_<your_name><CG><index_number>.ipynb.

Stack is an abstract data type that uses the Last In, First Out principle. One of the common applications is to implement as an undo method in other classes.

Undo methods are typically implemented using the Memento pattern or the Command pattern. In the Memento pattern, the series of current states are saved and can be retrieved later to replace the current state when undoing commands. In the Command pattern, the series of commands that have been run are saved, and the reverse commands are called to undo the commands.

The task is to:

- implement a `Stack` class
- use the `Stack` class to implement an undo method using the Memento pattern
- implement the `redo` method using the `Stack` class.

For each of the sub-tasks, add a comment statement at the beginning of the code, using the hash symbol '#' to indicate the sub-task the program code belongs to, for example:

```
In [1] : # Task 1.1
        Program code
```

Output:

Task 1.1

Write a `Stack` class, to implement a stack using an array. Include the following methods:

- `push(value)` inserts the value at the top of the stack.
- `pop()` returns the value at the top of the stack; if the stack is empty, return `None`.
- `is_empty()` returns a Boolean value `True` if the stack is empty; `False` if the stack is not empty
- `display()` displays the values in the stack in the following format:

```
Top of Stack
<Last Element Pushed>
    ⋮
<First Element Pushed>
```

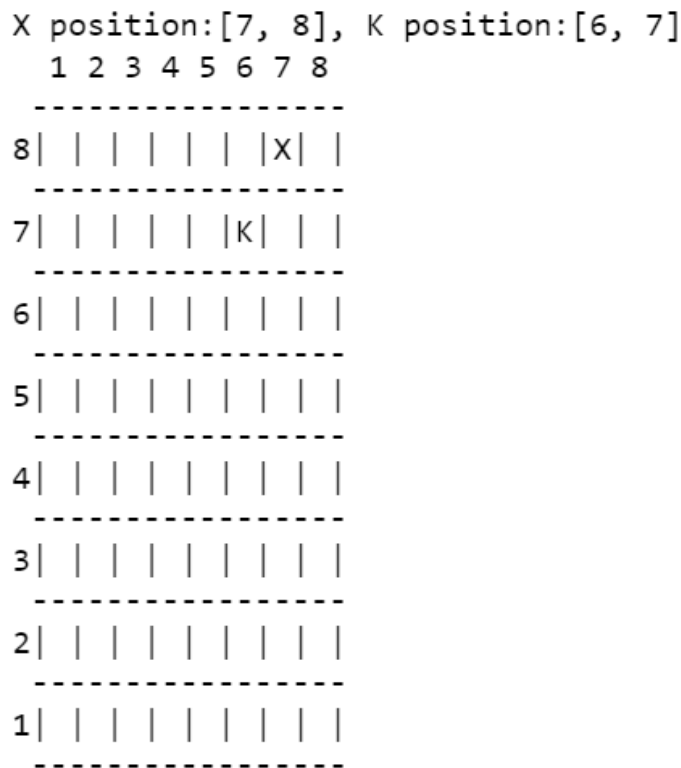
If the stack is empty, print "Stack is empty".

[6]

Task 1.2

The `random` library is built into Python. This library can be used to perform random actions such as random numbers. In particular, `randint(a, b)` is used to generate a random integer between `a` and `b`, inclusive of both.

An 8×8 chessboard consists of an 8×8 grid of small squares. For this task, the squares are numbered using a coordinate system as a list of two integers. The first integer is the column number, starting from 1 at the left, and the second integer is the row number, starting from 1 at the bottom. (See Figure 1 below)

**Figure 1**

A chess knight ($\mathbb{K}$) is a piece that occupies a square, and can then move to another square according to one of the following rules:

- moving two squares horizontally and then one square vertically in either direction, or
- moving two squares vertically and then one square horizontally in either direction.

The aim of the game is to let $\mathbb{K}$ occupy $\mathbb{X}$'s square.

The Knight class is designed as follows:

Knight
X_pos: List K_pos: List u_stack: Stack
Knight() grid() move(k_col: Integer, k_row: Integer) undo()

X_pos and K_pos are lists each containing 2 integers from 1 to 8. They represent the positions of K and X.

u_stack is a Stack object from **Task 1.1**.

Knight() will create an instance of the Knight class. It will randomly generate X_pos and K_pos. X_pos and K_pos cannot be identical lists during instantiation. It will also run the grid() method described below.

grid() will display Figure 1, depending on the values of X_pos and K_pos. It should also display the positions of K and X.

move(k_col, k_row) will check if the move from K_pos to [k_col, k_row] is a valid knight move. You may assume that the user will insert integers from 1 to 8. If the move is a valid knight move, the current K_pos is pushed into u_stack, and [k_col, k_row] is set as K_pos. If the move is not a valid knight move, it will display the message "Invalid move". It will also run the grid() method.

undo() will set K_pos as the previous K_pos list from u_stack. If there are no more moves to undo, it will display the message "No moves to undo". It will also run the grid() method.

Write a Knight class that implements the above-mentioned attributes and methods. [13]

Task 1.3

The redo() method can be implemented using a redo stack in the class. Whenever a move is undone, the current K_pos is also pushed into the redo stack, in addition to the implementation in **Task 1.2**.

When a move is redone, the current K_pos is pushed into u_stack and the previous undone state, retrieved from the redo stack, will be set as K_pos.

If a new move is made, the redo stack is cleared.

Write a subclass Knight2 that inherits from Knight. Knight2 will have a redo() method. Your code should also demonstrate appropriate polymorphism. [7]

- 2 A carpet company wishes to develop a web application to allow its website visitors to view its carpets according to the price, the country of origin and the design style.

The company has three outlets across the country.

The company manages its inventory in the database `CarpetsDetails.db`.

The following outlet information is to be recorded in the table `Outlets`.

- `OutletID`: a unique integer ID assigned to the outlet.
- `Location`: the area where the outlet is located.

The following carpets information is to be recorded in the table `Carpets`:

- `CarpetID`: a unique integer ID assigned to the carpet.
- `OriginalPrice`: the original price of the carpet e.g. \$1032.89.
- `Country`: the country in which the carpet was made e.g. Pakistan and Turkey
- `Style`: the design style of the carpet such as modern, contemporary, and traditional style.
- `OutletID`: a unique integer ID assigned to the outlet that sells the carpet.

The table `Promotion` should contain the following information to update the price of the carpet due to promotional offer and other seasonal promotions:

- `CarpetID`: a unique integer ID assigned to the carpet.
- `OutletID`: a unique integer ID assigned to the outlet that sells the carpet.
- `DiscountedPrice`: the price of the carpet after discount.

Task 2.1

Create an SQL file called `Task2_1_<your_name><CG><index_number>.sql` to show the SQL code to create the three tables in the database `CarpetsDetails.db`. [6]

Task 2.2

The files `Outlets.txt` and `Carpets.txt` contain information about the company's outlets and carpets respectively. The information in the two files is formatted in comma-separated lists.

For `Outlets.txt`, information about each outlet is given in the following order:

- `OutletID, Location`

For `Carpets.txt`, information about each carpet is given in the following order:

- `CarpetID, OriginalPrice, Country, Style, OutletID`

To celebrate the opening of its latest outlet, the carpets in the three outlets will be sold at the following discounts:

OutletID	Discount on the carpets
1	10%
2	10%
3	30%

Write a Python program to insert all the above required information into the database `CarpetsDetails.db`.

Save your program code as `Task2_2_<your_name><CG><index_number>.py`.

Run your program.

[7]

Task 2.3

Write a Python program and the necessary files to create a web application that allows the website visitors to filter the carpets according to their attributes.

Specifically, the web application should be able to:

- Receive inputs from a HTML form that allows user to:
 - select the:
 - country of origin of the carpets, and
 - design style of the carpets
 - and enter a discounted price range of the carpet. For example, a user may enter a price range between \$500 and \$1000.
 - Your program must display the discounted price in 2 decimal places.
 - The maximum price must be greater than or equal to the minimum price. If the user inputs otherwise, an appropriate error message is to be displayed and the user must key in again.
- Returns a HTML document to display only the carpets whose attributes fall within the above user's preferences.

Save your Python program as `Task2_3_<your_name><CG><index_number>.py`.

Save any additional files and subfolders in a parent folder named
`Task2_3_<your_name><CG><index_number>`.

[15]

Task 2.4

Run and test the web application using the following inputs:

- Country of origin of the carpets: Pakistan
- Design style of the carpets: Modern
- Price range: Between \$700 and \$3000

Save the output of the program as `Task2_4_<your_name><CG><index_number>.html`.

[2]